MINIMAX-H3 / TWO-GPU INFERENCE

Integrated Optimization Technical Guide

An implementation-level explanation of attention, INT8 row operators, TP2 communication, LoRA, memory residency and parallel VAEs.

TE	DiT	Video VAE
Group32 dense	Kitchen INT8	Kitchen + Q split

CURRENT SOURCE, NOT A BLENDED BENCHMARK CLAIM

Default INT32 SUM. Owned FP32 FC2 rotation. Optional sparse attention and scaled FP16 transport. Native conditioning, sampling and public VAE planning.

AUDITED 20 OCTOBER 2026 | 30 PAGES

Pinned ComfyUI: d49e888586dd8ae012c0667b33466b815fee07f7

Repository: 2xT4 minimax h3

C:/Users/Administrator/Documents/project/2xT4 minimax h3

Historical T4 results are scoped separately from the frozen local stack.

How to read this guide

A source-first explanation of what runs, why it helps, and what is still unproven.

The target is a two-rank MiniMax-H3 inference service for two T4 GPUs. The optimization is a combination of numerical operators, model partitioning, bounded weight movement and lifecycle control. Faster kernels alone do not explain its behavior.

Read in this order	Questions answered
Architecture and policy - pp. 3-5	Who owns the work? Which choices are defaults?
Attention - pp. 6-10	What is quantized? How do dropping and pooling differ?
Linear operators and LoRA - pp. 11-17	Why global scales and INT32 SUM? Where are buffers removed?
Loading and VAEs - pp. 18-25	How do disk, CPU, GPU windows and decode stages cooperate?
Preservation and validation - pp. 26-28	Which evidence applies? How should the system be operated?
Source catalogue - pp. 29-30	Where can each mechanism and measurement be checked?

Three distinct meanings of correctness

Same-method conformance means an optimized implementation matches its own numerical reference. **Native equivalence** means it matches the selected original operator at a defined boundary. **Generation quality** means trajectories, content, temporal structure and audio remain acceptable. Passing one does not imply the others.

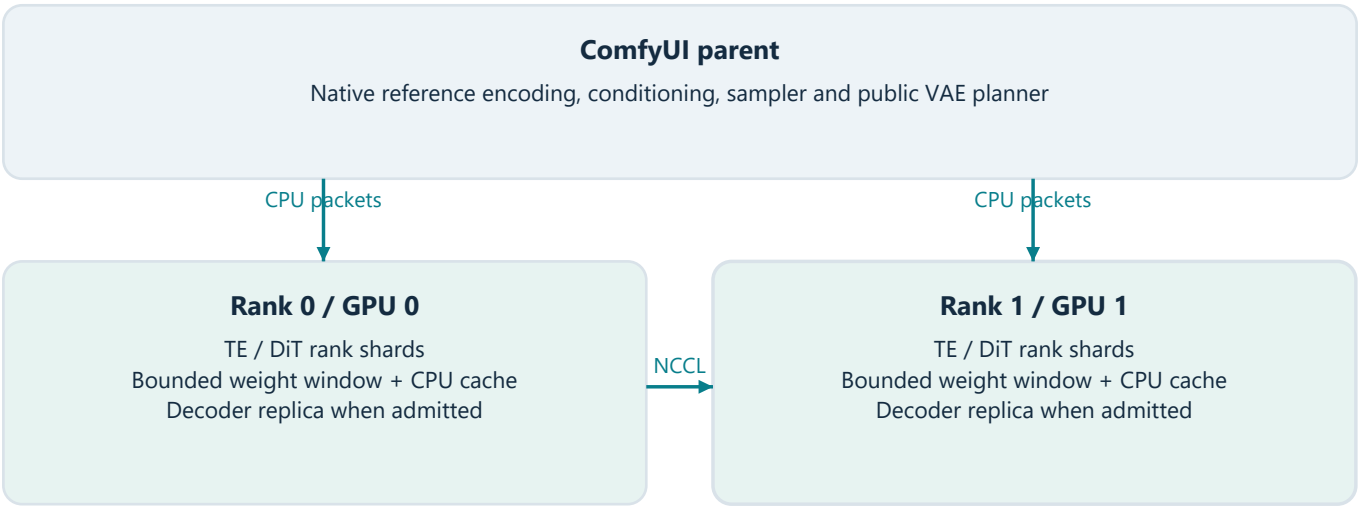
> **Reading the figures**
> Teal marks a current default or active work. Amber marks an optional approximation or policy. Grey marks Native behavior, preserved control flow or inactive storage. Diagrams are explanatory schematics unless a chart explicitly names a measurement.

Notation: S is the complete packed token count, M the flattened row count, K the linear input width, N the linear output width, D the attention head dimension, and r a TP rank. MiB = 2^20 bytes; GiB = 2^30 bytes. Kernel time, operator time and request time have different boundaries.

Source: S01, S03, S30, S33-S39

One Native pipeline, two persistent ranks

Keep ComfyUI semantics in the parent; distribute the expensive model work.



One session lock and an explicit phase protocol coordinate both owners.

The ComfyUI parent retains prompt/image/audio preprocessing, public model interfaces, sampler scheduling and public VAE planning. A session owns two persistent workers, each bound to one CUDA device. Weights can remain in bounded CPU caches while GPU weight windows change between TE, DiT and decode phases.

Boundary	Ownership and consequence
Parent to ranks	Bounded synchronous CPU packets preserve tensor dtypes, nested containers and PackedLayout metadata. Persistence amortizes startup; this is not an asynchronous transport-overlap claim.
Between ranks	Actual distributed collectives complete the TP row projections and VAE exchanges. The workers share the same configuration and collective order.
Phase transition	TE, sampling and VAE decode have explicit lifetimes. Incompatible phases release or fence their owners before another stage is admitted.

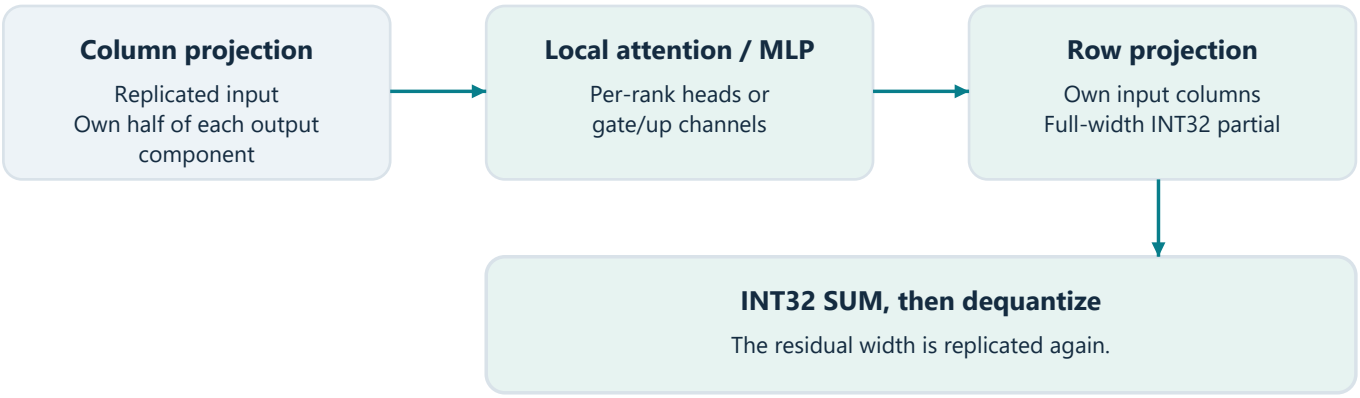
Workers set NCCL_P2P_DISABLE=1 and both minimum/maximum channel counts to 4, leaving NCCL protocol selection at its default. Direct GPU peer copies are not assumed. Those settings are a runtime contract, not a measurement of physical PCIe bandwidth.

> **Important limit**
> One-GPU logical-rank previews test arithmetic and wiring. They do not validate physical NCCL behavior, two-T4 latency or concurrent-request throughput.

Source: S03-S05, S23-S26

TP2 changes the shapes that matter

Partition each packed component correctly, then recombine row-projection sums.



Per-rank quantity	TE / Qwen3-VL	DiT main blocks
Hidden / output width	5120	5376
Attention heads; D	32 Q, 4 KV; D=128	28 Q/K/V; D=128
Local Q or attention width	4096	3584
Local FFN activation width	12800	7168
Gate/up representation	Separate tensors	Packed local width 14336
Row O: local K -> N	4096 -> 5120	3584 -> 5376
Row FFN: local K -> N	12800 -> 5120	7168 -> 5376

Column-parallel projections split output rows. DiT QKV is three components, and FC1 is two: each rank owns half of each component, not a single half of the concatenated tensor. Row-parallel projections split the input dimension and leave the full output width on both ranks.

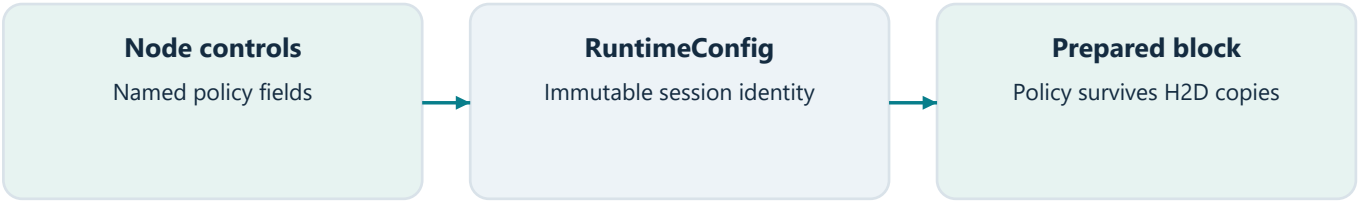
For a row projection, each rank computes a local INT32 partial P[r]. The output is reconstructed from SUM(P[r]) using the shared activation scale and output-channel weight scale. RMSNorm and DiT AdaLN are replicated. Sequence lengths vary with Native packing; a historical S=6814 fixture is not a fixed deployment contract.

Source: S06, S07, S16, S21

A per-session operator policy

Backend selection is explicit and becomes part of cache and worker identity.

Setting	Current default	Alternative / scope
TE attention	tilelang_group32	native
DiT main attention	kitchen	tilelang_dense / tilelang_drop / tilelang_scalar / native
Token Refiner	Native SDPA	Independent of DiT main attention mode
Row-linear backend	tilelang_packed	native / tilelang_scaled_fp16
FC2 quantizer	owned_fp32	native full-width rotation
SwiGLU padding / selective B	True / True	Each has eligibility guards
Scalar score cap	96 MiB per rank	0 selects inline scalar summaries
LoRA loading	online	offline baking
Adaptive lookahead / idle	False / False	Explicit runtime options
VAE decode	auto	Per-VAE native override



Different policy -> different service/cache identity; no stale backend reuse.

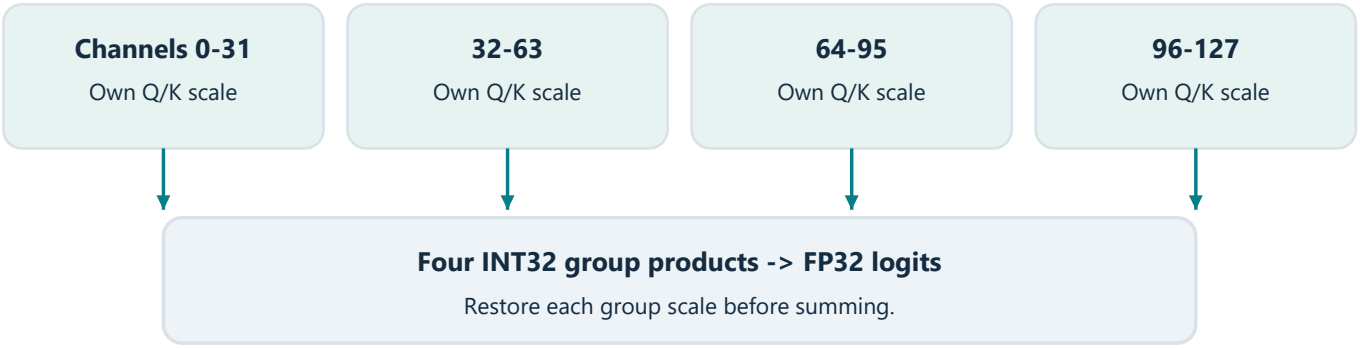
The loader constructs an immutable H3OperatorPolicy and RuntimeConfig. Configuration equality decides whether the persistent service may be reused. Operator identity also follows prepared blocks, residency copies and idle-prefetch cache keys; changing a backend must not reuse a block configured for another backend.

Do not confuse two settings
The default native_int32 reduction contract remains in RuntimeConfig. Selecting tilelang_scaled_fp16 is a separate explicit backend choice. The fc2_quantizer preference only affects eligible packed rows with SwiGLU fusion; it does not override the scaled-FP16 backend.

Source: S01-S03, S13, S14, S18

What Group32 INT8 attention computes

Fine-grained Q/K scales, an INT32 dot product, and an FP16 probability-value path.



For each token and each 32-channel group g, a scale is computed from the absolute maximum. Q is quantized directly. K is centered first. With D=128, the dot product consists of four 32-channel INT8 products, each restored with its own pair of FP32 scales.

```
sQ[i,g] = max(abs(Q[i,g])) / 127, floored at 1e-8
QI[i,g] = clamp(round(Q[i,g] / sQ[i,g]), -127, 127)
logit[i,j] = sum_g INT32(QI[i,g] dot KI[j,g])
             * sQ[i,g] * sK[j,g] / sqrt(D)
```

K centering uses 64-key leaf centroids and the unweighted mean of those centroids, including the ragged final leaf with its actual count. Subtracting one common K offset changes every logit in a query row by a common constant in exact arithmetic, so softmax is invariant. Quantization and finite-precision boundaries still introduce approximation.

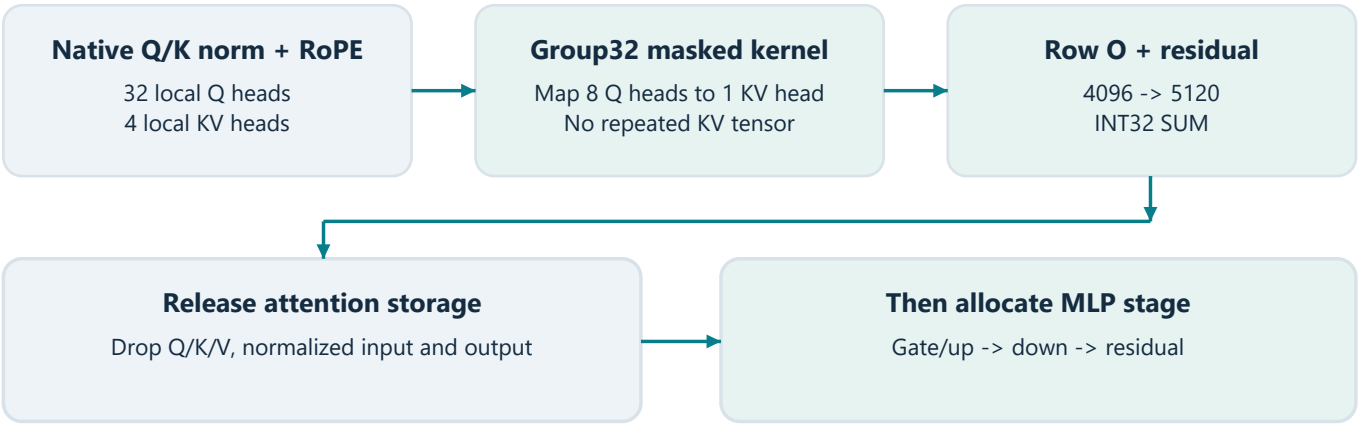
Online softmax maintains a running maximum, denominator and weighted numerator as each 64-key leaf arrives. It avoids allocating a full S-by-S score tensor. Softmax state and output accumulation use FP32; probabilities and V feed an FP16 PV GEMM. The input preparation retains the selected FP16 rounding boundary.

> **Precision meaning**
> Group32 offers a more local Q/K representation than coarser scaling, but this document does not claim universal accuracy superiority over Kitchen. FP16 PV is the integrated TileLang path; the experimental INT8-PV attention variants are not enabled here.

Source: S08-S10, S33

TE: compact GQA, real masks, short lifetimes

Preserve Native embeddings, Q/K normalization and RoPE before the attention boundary.



Each rank has 32 Q heads but only 4 K/V heads. A query head maps to its shared KV head inside the TileLang kernel, avoiding explicit replication of KV storage. This is grouped-query attention (GQA), not a change to the learned projections.

Contract	Implementation behavior
Mask shapes	2D, 3D and 4D masks must broadcast to the local BHND query/key shape. Boolean masks become additive 0/-infinity values on device.
Online softmax	Masks enter the exp2-domain logits. A fully masked query returns zero. A masked early leaf does not prevent later valid leaves from contributing.
Native fallback	Unsupported architecture logs a fallback. Native backend selection preserves the full-model size policy, including short-prompt masked GQA behavior.
Scope	TE prefill only. Past-KV-cache execution remains rejected.

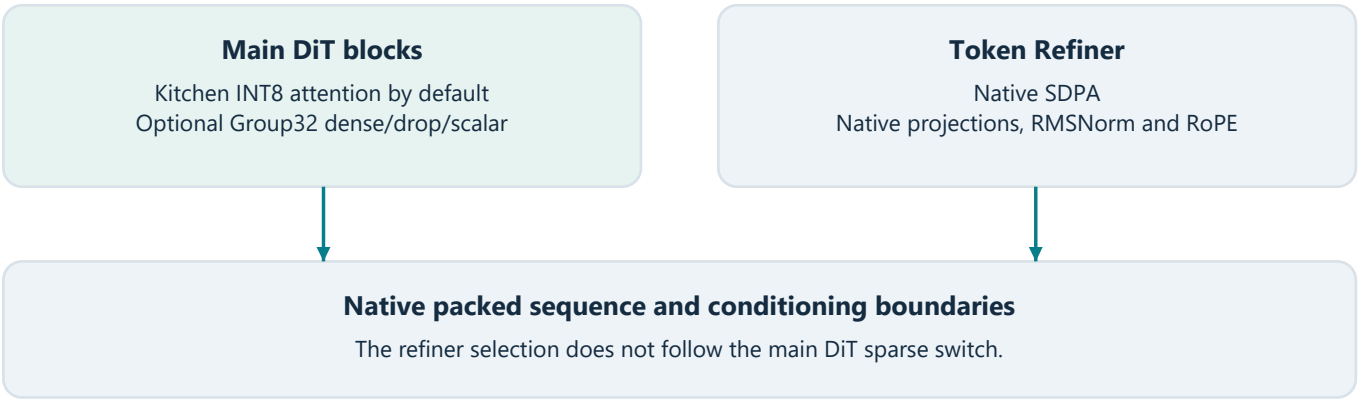
After the attention residual accumulation, normed input, Q/K/V and attention output references are deleted before allocating MLP intermediates. After down projection, gate/up/normalized inputs are released before the MLP residual, followed by its output. This changes reference lifetimes, not residual arithmetic order.

> **Why explicit deletion helps**
> A view or closure can keep an entire storage alive. Removing only one alias is insufficient. In inference, releasing the last Python owner permits allocator reuse after stream safety is satisfied; autograd outside this runtime can retain its own references.

Source: S06, S08, S34

Kitchen attention is the main DiT default

The Token Refiner deliberately retains Native SDPA.



Main DiT blocks normalize and modulate their inputs, project local Q/K/V, apply Native Q/K norms and RoPE, then dispatch through the operator policy. The default calls comfy_kitchen.int8_attention. Its result is converted to token-major local features before the row-parallel O projection and residual update.

Kitchen and TileLang Group32 are different numerical methods. Kitchen remains the selected DiT and Video VAE attention implementation; the optional TileLang dense path uses the Group32 QK / FP16 PV contract. Historical experiments with alternative PV precision or centering do not silently change this selection.

The refiner is a separate boundary

NativeRefinerAttention wraps the existing refiner module rather than storing bound methods that could still point at canonical CPU weights after residency copying. It keeps the Native projections, RMSNorm and fused RMSNorm/RoPE behavior, and explicitly calls PyTorch scaled_dot_product_attention. Refiner tokens are never routed through sparse DiT attention.

> **An old name is not an old execution path**
> KitchenRefinerAttention remains only as an import-compatible alias for the Native SDPA wrapper. It no longer selects INT8 attention. This deliberate change is not byte equivalence with the former INT8 refiner.

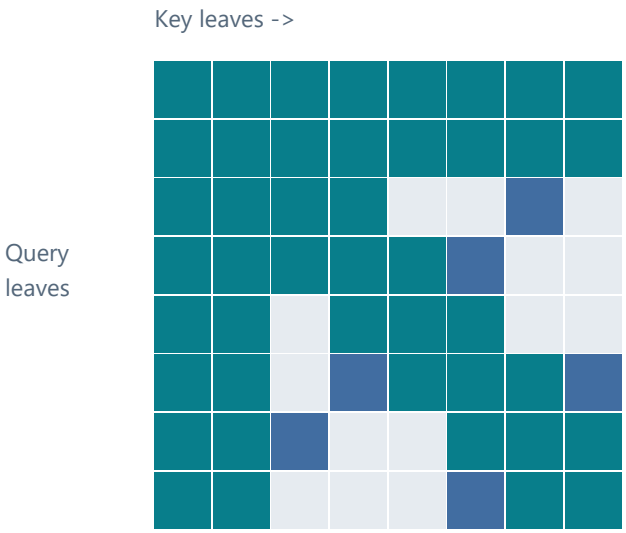
For the optional dense Group32 DiT kernel, the current caller uses a 64-query tile, 128 threads, a row warp policy and cached Q operands. Such launch/resource choices are architecture-specific implementation details, not independent guarantees of Tensor Core utilization or pipeline speedup.

Source: S01, S07, S09, S33, S34

Which DiT attention blocks may be omitted?

Route 64-token leaves; protect conditioning through the current Native layout.

Not a captured route map.



Protected / mandatory

Prefix rows + columns
Neighbor leaves

Other video pairs

Blue: threshold kept
Grey: omitted

Illustration: 2 prefix leaves + 6 generated-video leaves

The sparse modes require DiT self-attention with matching Q/K/V shapes. Query and key tokens are grouped into leaves of 64. A query-leaf centroid is compared with each centered key-leaf centroid; tau0 retains scores above the query-dependent mean threshold. Nearby leaves and the first key leaf are mandatory.

The runtime obtains the dense prefix from the current PackedLayout. All text, references, conditioning and generated audio before the final generated-video segment remain protected. A missing or stale layout is rejected, not replaced with the historical 874-token fixture constant. Protection rounds to whole leaves, so a leaf intersecting the prefix is retained as a whole.

Always retained	Policy consequence
Prefix query leaves	Attend all key leaves.
Prefix key leaves	Remain visible to all query leaves.
Local neighborhood / first key leaf	Provide the routing method's mandatory connectivity.
Other video-video pairs	Centroid threshold decides whether full token work is kept.

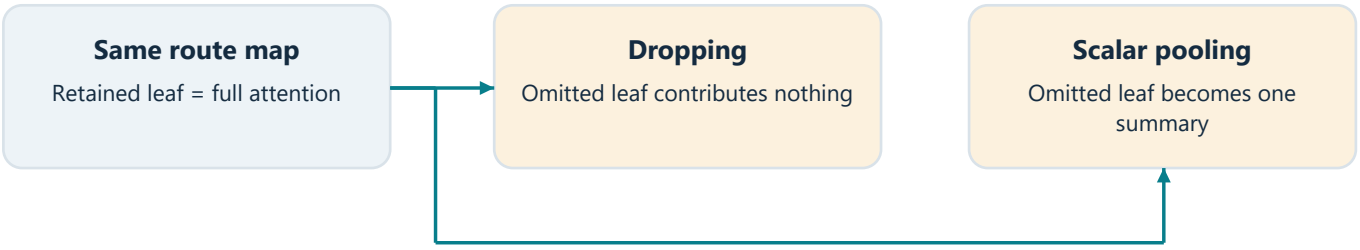
Sparsity is not speedup

Routing, preparation, launch overhead and work on retained blocks remain. Scalar pooling also processes omitted-leaf summaries. A percentage of omitted blocks is not a measured percentage of request-time savings.

Source: S01, S10, S11, S38

Dropping and scalar pooling are different

Both are optional zero-shot approximations; retain both choices.



Pooling preserves approximate leaf mass, not token-level attention structure.

For an omitted leaf	Dropping	Scalar pooling
Logits / denominator	No contribution	One centroid logit plus log(actual token count)
Value contribution	No contribution	Weighted mean V of the omitted leaf
Retained leaf	Full Group32 QK and FP16 PV	Same full Group32 QK and FP16 PV
What is lost	All omitted probability mass	Within-leaf logit/value variation and correlation

```
pooled_logit[i,b] = dot(Q[i], K_centroid[b]) / sqrt(D)
                  + log(count[b])
pooled_numerator += exp(pooled_logit - running_max)
                  * V_centroid[b]
```

The log-count term represents multiple tokens with one summary; simply adding one unweighted centroid token would change the intended mass. For a ragged leaf, count is the true number of keys. The FP16 pooled-probability boundary and streaming normalization remain part of this approximation.

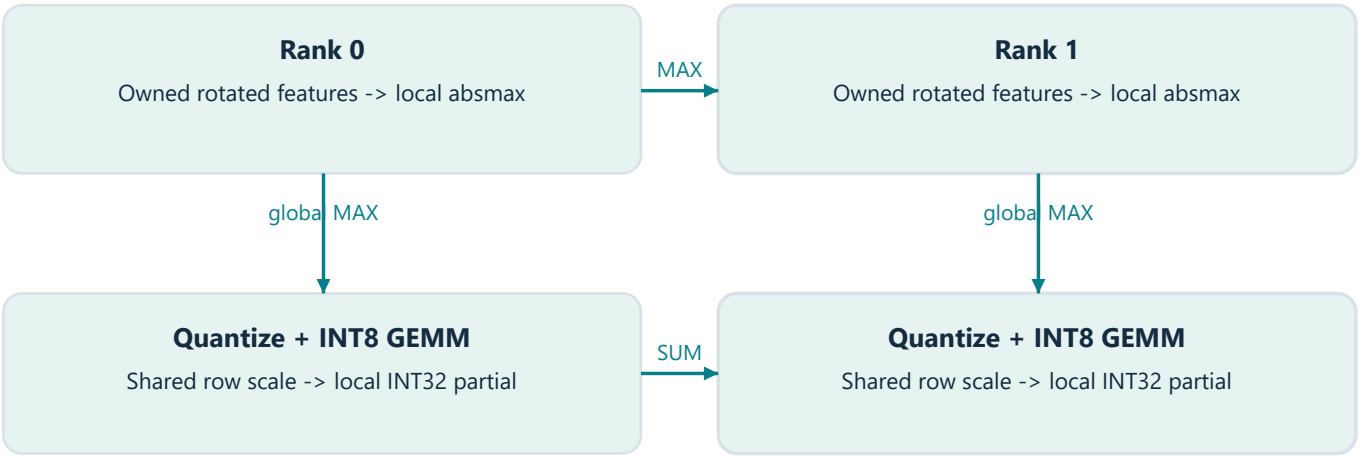
Scalar mode may precompute only omitted-leaf query-centroid scores. The workspace is B*H*Np*Nb*4 bytes per rank. A retained real H3 fixture requires 82,066,432 bytes (78.3 MiB), below the 96 MiB default cap. A zero cap or a larger required workspace selects inline summaries. This trades one preparation kernel and memory for less centroid work in the main kernel.

> **Quality boundary**
> Sparse implementation conformance is checked against the same routing/pooling method. Sparse-versus-dense L1/L2 is not a generation-quality gate. Semantic/distribution drift and temporal behavior require full-pipeline generation review, which is outside these integrated operator checks.

Source: S01, S10, S11, S38

Global row scales make TP INT8 coherent

Local integer GEMMs are only valid together with the right cross-rank contract.



SUM is over INT32 partials; dequantization follows the collective.

```
m[i] = MAX_r max(abs(rotated_X[r][i,:]))
sA[i] = max(m[i] * float32(1/127), 1e-30)
Q8[r] = clamp(nearbyint(rotated_X[r] / sA), -128, 127)
P[r] = Q8[r] @ W8[r].T                  # INT32
Y[i,j] = float32(SUM_r P[r][i,j]) * sA[i] * sW[j]
```

The maximum must cover the entire logical activation row, even though each rank owns only part of it. Independent half-row scales would make the integer partials incompatible. Per-output weight scales also retain the full-width checkpoint quantization semantics; they are not recomputed independently on input-column shards.

The packed row path optimizes ownership padding and activation preparation. It still uses Kitchen's native INT32-accumulating INT8 GEMM and an INT32 AllReduce SUM before dequantization. Here, packed does not mean compressed INT8 network transport.

ConvRot groups contain 256 channels and are fully owned locally. The row quantizer follows its own Kitchen-compatible floor, fast division, nearby rounding and [-128,127] saturation. This is distinct from attention's Group32 quantizer with floor 1e-8 and [-127,127] range; interchanging them would change the numerical method.

Source: S12, S13, S16, S17, S21

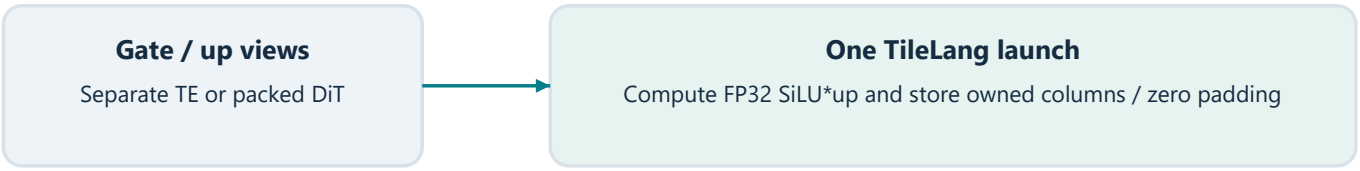
Fuse SwiGLU with ownership padding

One output write replaces eager activation and padding stages.

Before / full-width preparation



Fused path



SwiGLU computes silu(gate)*up. TE supplies separate gate and up tensors; DiT supplies strided halves of one packed FC1 result. The fused TileLang kernel reads either layout directly. It writes the owning rank's output columns and explicitly writes positive zero into unowned columns.

The full-width mode feeds the same Native FP32 ConvRot GEMM geometry. It preserves FP32 exp, rounded addition/division and multiplication; it does not substitute a fast approximate sigmoid, FP16 activation or a different row quantizer. With eligible shapes, one launch replaces eager SwiGLU plus padding work and removes the separate activated local tensor.

Site	Input / local activation	Padded output
TE down	Separate gate/up, each M x 12800	M x 25600 FP32
DiT FC2, full-width mode	Packed M x 14336; local activation 7168	M x 14336 FP32
DiT FC2, owned mode	Same packed input; local activation 7168	M x 7168 FP32, no global padding

Fusion boundary
This removes intermediate traffic and launches around activation preparation. It does not fuse the INT8 GEMM, MAX collective, SUM collective and residual into a single kernel. Complete-row and full-request effects must be measured at their own boundaries.

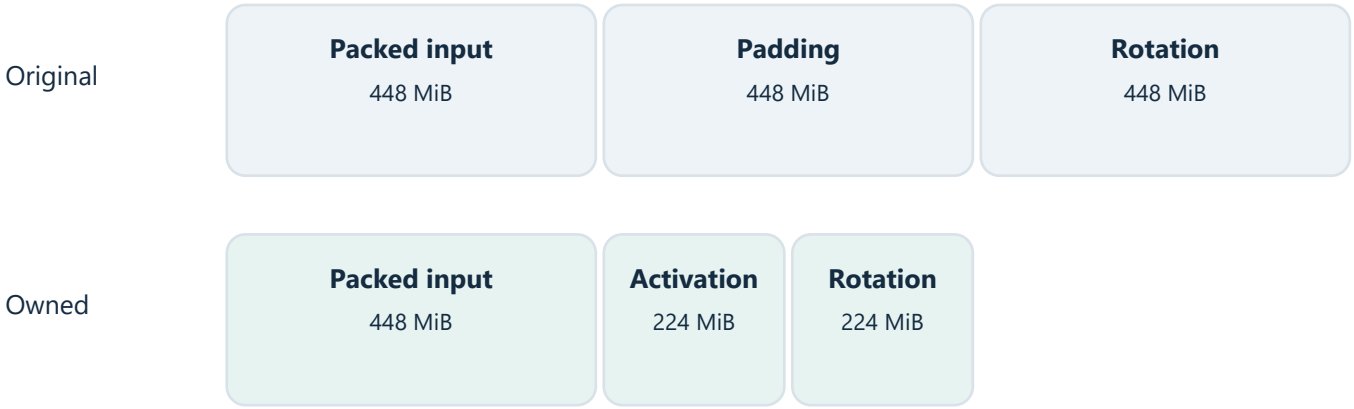
The shared helper also releases the full padded tensor in the actual caller after its last rotation read. An inner helper's del cannot free a buffer still referenced by the caller. Same-stream last-use ordering is retained.

Source: S12, S13, S30, S34

Rotate only the channels the rank owns

The current DiT FC2 default removes full-width FP32 temporaries.

Illustrated live payload at S8192, per rank (not total VRAM)



At quantization the original also retains 56 MiB INT8 + row vectors.

At S8192, each rank's packed FC1 output is 448 MiB. The original path overlaps it with 448 MiB of zero padding and 448 MiB of full-width rotation. Padding was also retained through quantization, where the owned INT8 output adds 56 MiB. Repairing lifetime alone reduces that latter overlap.

Owned FP32 rotation materializes only the local 7168-channel SwiGLU activation (224 MiB) and its rotation (224 MiB). All local 256-channel ConvRot groups are complete. The same global FP32 MAX, INT8 rounding, learned weight scales and INT32 SUM remain. No lossy activation conversion or compressed transport is introduced.

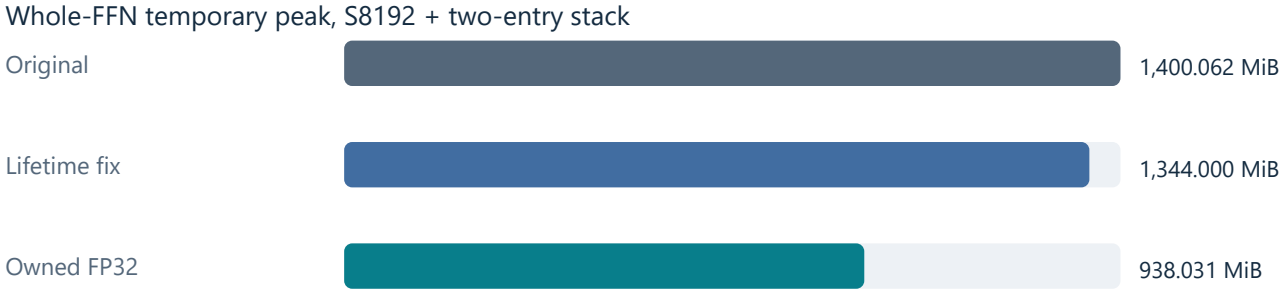
> **The arithmetic caveat**
> The flattened M dimension of the FP32 rotation GEMM changes. Algebraic group locality alone cannot prove identical rounding across algorithms/backends. The trained qualification matrix observed identical payload, scale bytes and complete FC2 outputs; this is not a universal bit-exactness theorem.

Eligibility is deliberately narrow: packed H3 FC2 width 14336, FP32, TP2, group size 256, packed row backend and enabled SwiGLU fusion. Other shapes/backends retain their existing path. TE down keeps the full-width rotation with the lifetime repair. Set fc2_quantizer=native to restore full-width geometry.

Source: S01, S13, S30-S32

FC2: memory reduction survives integration

Actual historical two-T4 operator measurements, not an end-to-end forecast.





The charts use S8192 with the retained two-entry LoRA configuration and its existing optional Row Partial mode. Fourteen synchronized wall samples span two reversed rounds; the slower rank is selected per iteration. Temporary memory is measured above the resident entry in separate peak passes. It excludes allocator reservations and NCCL internal allocations.

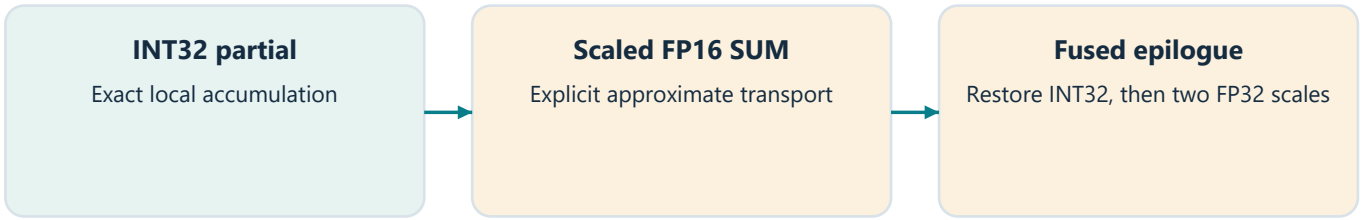
The owned path reduces whole-FFN temporary peak by 33.00% and the pooled FFN median by 5.44% in that allocation. The peak moves to unchanged base FC1 dispatch, whose two output buffers plus INT8 activation/scales total 938.031 MiB. FC2 itself is no longer the largest explicit FFN boundary.

Scope and version
These physical results used the historical T4 software stack. They do not qualify the current frozen conda stack on Linux. S32768 integrated confirmation also passes byte gates, but its two-entry whole-FFN timing varies by round: -0.56% then +5.02% reduction. Do not advertise its pooled gain as stable.

Source: S30-S32, S29, S39

Scaled FP16 SUM with a fused epilogue

Trade communication precision explicitly; retain Native INT32 as the default.



Default remains native INT32 SUM

The optional backend changes representation, not merely the epilogue.

```
shift = max(0, ceil(log2(2*Klocal*127*127 / 60000)))
payload[r] = FP16(float32(P[r]) * 2**(-shift))
reduced = AllReduce_SUM(payload)
restored = INT32(float32(reduced) * 2**shift)
Y = (float32(restored) * activation_scale) * weight_scale
```

The power-of-two factor uses a worst-case bound and adds no extra MAX collective. FP16 halves the logical per-rank SUM payload from four to two bytes per element. That is payload accounting, not a guarantee of half the wire traffic, collective duration or total operator latency.

The TileLang epilogue reconstructs the specified INT32 value and performs the two left-associated FP32 scale products in one kernel, avoiding a separate reconstructed INT32 allocation. Current support requires TP2 FP32 input, aligned local groups, INT8 weights and contiguous vector FP32 weight scales on physical SM75. Unsupported contracts fall back before collectives.

Cancellation makes this approximate

Let P0=2^shift*2049 and P1=-2^shift*2048. The exact sum is 2^shift; FP16 encoding can round both magnitudes to 2048, producing zero. Agreement with an independent FP16 codec reference proves implementation correctness, not equality to Native INT32 SUM or preservation of generation semantics.

Source: S01, S14-S16, S34, S38

Selective B removes unowned FC1 outputs

Keep adapter order, separate GEMMs, and post-B scaling.



B output row order is [gate | up], not [all rank-0 | all rank-1].

Online LoRA adds a floating-point bypass around the INT8 base projection. For each adapter, hidden=linear(x,A), delta=linear(hidden,B), then multiply by its effective strength and add to the target. Adapters execute in Native order. Strength is not folded into B, and A factors are not concatenated into a different GEMM.

Selective B is enabled only for qualified main DiT FC1 geometry: A=[128,5376], B=[28672,128]. Each rank needs 7168 gate rows and 7168 up rows. Those rows are read and packed once on CPU, so B produces 14336 owned outputs rather than all 28672. Unqualified shapes, QKV, refiner sites and other targets keep full B behavior.

LoRA choice	Semantics / status
Selective FC1 B, default on	Skips unowned B rows for the guarded geometry; validated selected-output arithmetic, not a general GEMM-rounding theorem.
Native row-input gather, default	Assemble the complete input before A, retaining Native A GEMM geometry.
Row A partial reduction, optional	Compute A partials locally and SUM the small hidden. Changes FP32 reduction grouping, so remains an explicit approximation.
Concat A	Retired; requesting it raises a clear error.

Row Partial staging is bounded to 32 MiB of small hidden tensors and only used when this can replace a larger SwiGLU activation. Neither selected B nor bounded staging changes the base INT8 GEMM or enables approximate SUM for that base projection.

Source: S01-S03, S18, S30, S33

Bake whole layers across two GPUs

Move repeated inference adapter work to a reusable checkpoint conversion.



Offline mode first releases the previous TP service. A size-balanced assignment gives each affected DiT/refiner layer to one of up to two spawned GPU workers. Each worker processes one full-width weight matrix at a time; a matrix and its quantization scale are never split between GPUs.

For an INT8 target, the worker performs Native FP32 dequantization, ordered adapter merging, and deterministic full-input-width ConvRot requantization. Dense targets return to their original storage dtype. Quantizing the full width before later TP splitting is essential: independent half-width weight maxima would change the model.

The base checkpoint is copied once to a new scratch artifact. Workers write disjoint weight/scale ranges through independent descriptors and fsync their writes. The parent publishes only after all workers finish, with source/stack identities, output signature, assignments and checksums in the receipt. No merged weight tensors travel over NCCL or parent pipes.

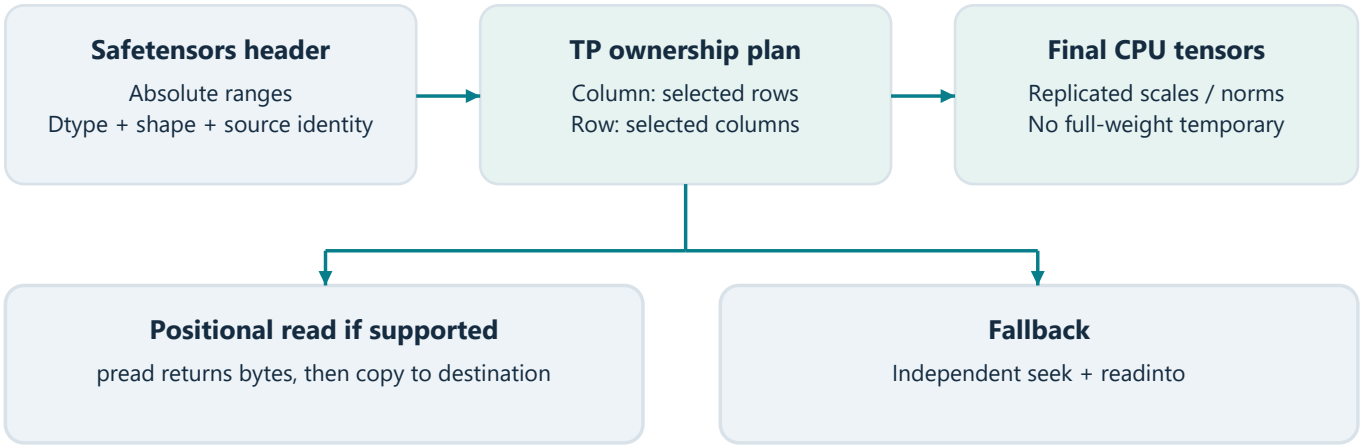
> **Warm reuse and numerical meaning**
> The cache key binds the base and ordered adapter stack, including strengths. A valid baked checkpoint is reused and the online stack is cleared to prevent double application. Baking introduces a new requantization boundary; it is not byte-exact with online LoRA over the original INT8 weights.

Cancellation is checked during copying and between matrices. Failure removes only unpublished scratch owned by this invocation. A new bake needs space for another full diffusion checkpoint. Baking/loading costs are one-time preparation costs, not steady-state inference latency.

Source: S02, S18, S19, S35

Read owned ranges, not full temporary weights

CheckpointReader supports positional access without changing the model schema.



The safetensors header maps each tensor to a dtype, shape and byte interval. The TP plan reads owned output rows for column projections, owned input columns for row projections, and replicated scales/norms where required. Current files retain the qualified INT8 ConvRot metadata. Source size and modification-time signatures guard against changing a checkpoint during a planned request.

Path	What it actually does
os.pread, when supported	Read at an absolute byte offset, loop on short reads, and copy returned bytes into the final tensor in chunks up to 8 MiB. It removes shared file-cursor mutation.
Seek + readinto fallback	Used when pread is unavailable or a descriptor rejects it, including ordinary Windows operation. Correct destination offsets and range order remain.
Rank-local offline export	Optional contiguous per-layer/rank safetensors with Native state hashes and source identity. This changes storage layout, not tensor math.

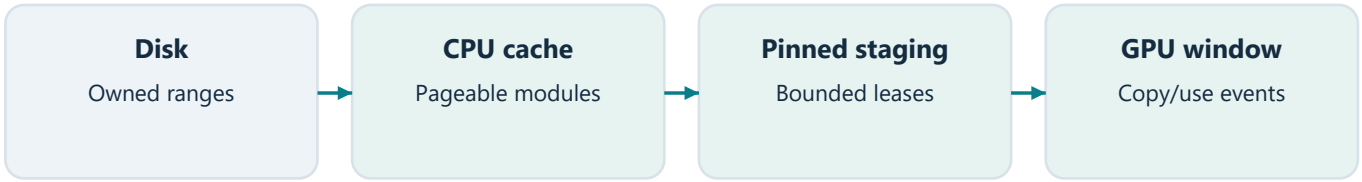
Pread is not direct I/O
Python os.pread returns a bytes object; it cannot fill a caller-supplied tensor buffer. This path is neither zero-copy nor O_DIRECT and does not bypass the operating-system page cache. No local SSD result is substituted for T4 storage behavior.

CPU decompression is currently a separate experiment. ZSTD, BitShuffle and FSE benchmarks are not installed in the active loader, so the diagrams do not place a decompressor in the production critical path.

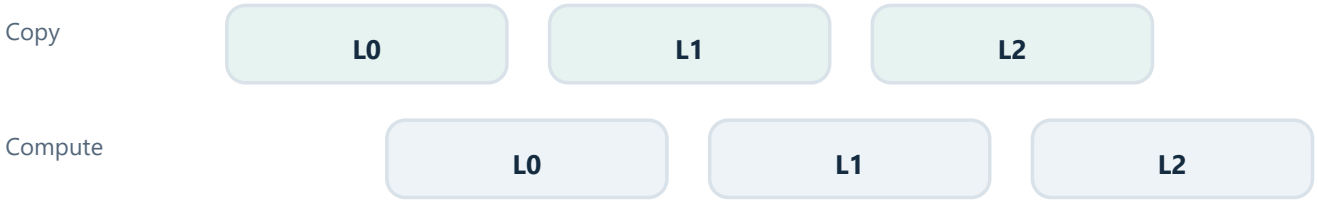
Source: S20-S22, S34

Bound the pipeline from disk to GPU

Pageable cache, pinned staging and GPU windows have separate owners and limits.



Planned overlap (schematic, not a profiler trace)



A copy must complete before use; eviction waits for last-use safety.

HostWeightCache prepares immutable CPU modules under a process-shared HostBudget. Per-rank cache capacity limits retained pageable modules, while construction reservations cover preparation peaks. A cache hit can bypass disk/CPU preparation without implying that the module is already pinned or resident on a GPU.

ResidencyWindow admits pin staging, copies modules on a CUDA copy stream, waits on copy events before use, and delays storage retirement until recorded compute reads are safe. Eviction considers future uses and explicit retained keys. CPU and GPU lookahead are scheduling horizons, not permissions to exceed their byte budgets.

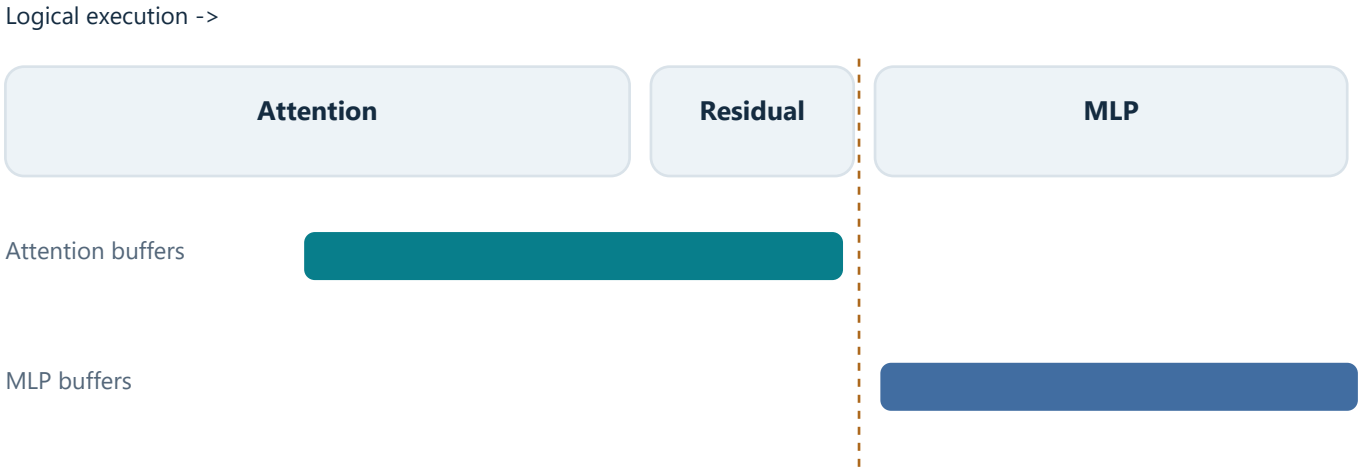
RuntimeConfig default	Meaning
GPU weights: 6 GiB/rank	Live/pending managed weight bound; does not include all activations or library state.
Pinned: 6 GiB/rank	Admission bound for staging, with allocator-aware accounting.
Pageable cache: 3 GiB/rank	Retained prepared CPU weight cache.
Shared host: 16 GiB	Cross-rank managed reservations, not total machine RSS.
CPU/GPU horizon: 16 / 2	Bounded upcoming preparation and distinct GPU groups; reserve remains enforced.

Adaptive lookahead defaults off. When enabled, the GPU depth becomes one or two upcoming distinct groups based on driver free memory plus reusable allocator memory, the weight cap and reserve. Admission rechecks headroom immediately before every copy. The opt-in also retains the existing CPU timing adaptation.

Source: S03, S20, S22, S23, S34

Release stages, not just local variables

The last owner determines when a large tensor can leave the live set.



Release last attention owners before creating the MLP live set.

Peak storage is determined by overlap. Summing the largest buffer in each stage overstates a peak if those buffers never coexist; counting only returned tensors understates it when views, closures, caller arguments or autograd retain a backing allocation. The code therefore gives attention, MLP, preprocessing and VAE phases explicit release boundaries.

Release point	Why it exists
TE attention residual	Drop normalized input, Q/K/V and attention output before the MLP starts.
DiT attention residual	Release attention views, packed QKV and projection temporaries before FC1/FC2 work.
After FC2 rotation	The caller drops padding after its last read; helper-local deletion alone cannot release caller ownership.
Encoder to sampling / decoder	Release staged encoder model, patcher and host lease before admitting the next large phase.
Close / abort	Fence/release workers, parent leases and idle speculation; unregister closed-service atexit callbacks.

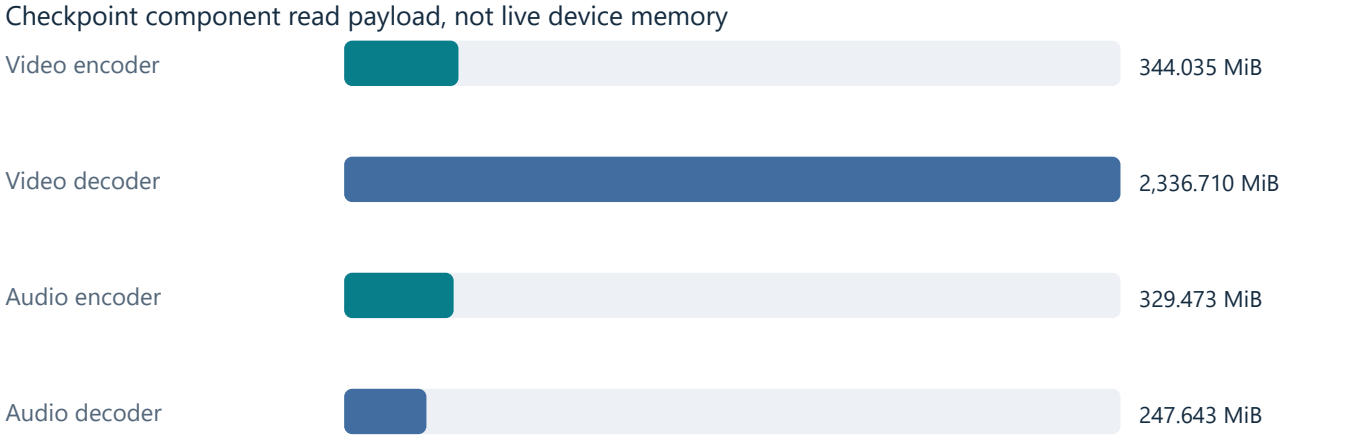
What memory numbers mean
Named tensor payload, PyTorch allocated peak, allocator reserved peak, managed host reservation, process RSS and device/NVML usage are different quantities. The FC2 chart reports scoped PyTorch temporary allocation; it is not a 33% claim for full-pipeline VRAM.

The owned-FC2 ledger moves a representative block-created payload maximum to the O projection, where attention buffers remain alive during the row call. Optimizing one stage can expose another bottleneck.

Source: S03, S06, S07, S13, S23, S24, S30

Load the encoder first, decoder only at output

Native public planning remains; the unused model half has no parameter storage.



MiniMaxH3ParallelVAELoader recognizes Native H3 video/audio checkpoint signatures and creates a StagedVAE. A meta-device shell retains Native metadata and public methods. At initial load, only encoder-side parameters and shared latent normalization are read. The inactive decoder is removed before real storage allocation.

Before denoising, registered encoder stages are explicitly released. At first decode, the encoder patcher, model backing and host lease are torn down before decoder reads and construction. A later encode reloads the encoder automatically. The session lock and VAE lock serialize these switches.

Stage	Retained checkpoint components
Video encode / decode	encoder + quant_conv / decoder + post_quant_conv
Audio encode / decode	encoder + pre_block + posterior projections / decoder + dec_in_proj
Shared / reconstructed	Latent normalization, Native pixel statistics, deterministic nonpersistent decoder RoPE and unit qk_norm_scale buffers.

Chart boundary
Bars show checkpoint payload bytes read for each component, with shared latent buffers included in both stages. They are not peak RAM/VRAM. Parent GPU transfer/offload remains under Native model management; worker decoders use a separate conservative admission.

Source: S02, S03, S24, S35

Query parallelism keeps global context

Split query ownership, exchange all keys and values, join only final projections.



The video decoder uses all 36 trained transformer blocks. Both ranks hold decoder weights; token/query ownership is split contiguously at a 128-query scale-group boundary. The split includes the five appended register/padding positions. Rank 0 performs Native prefix projection once and broadcasts the other owner's token range. RoPE uses the original global coordinates.

At each attention layer, both ranks project their local Q/K/V. Local Q is kept, while K and V are gathered globally. Each owner therefore attends over the entire key/value context with Kitchen INT8 attention. After the last decoder block, projected outputs are joined and Native patch reconstruction is completed on rank 0.

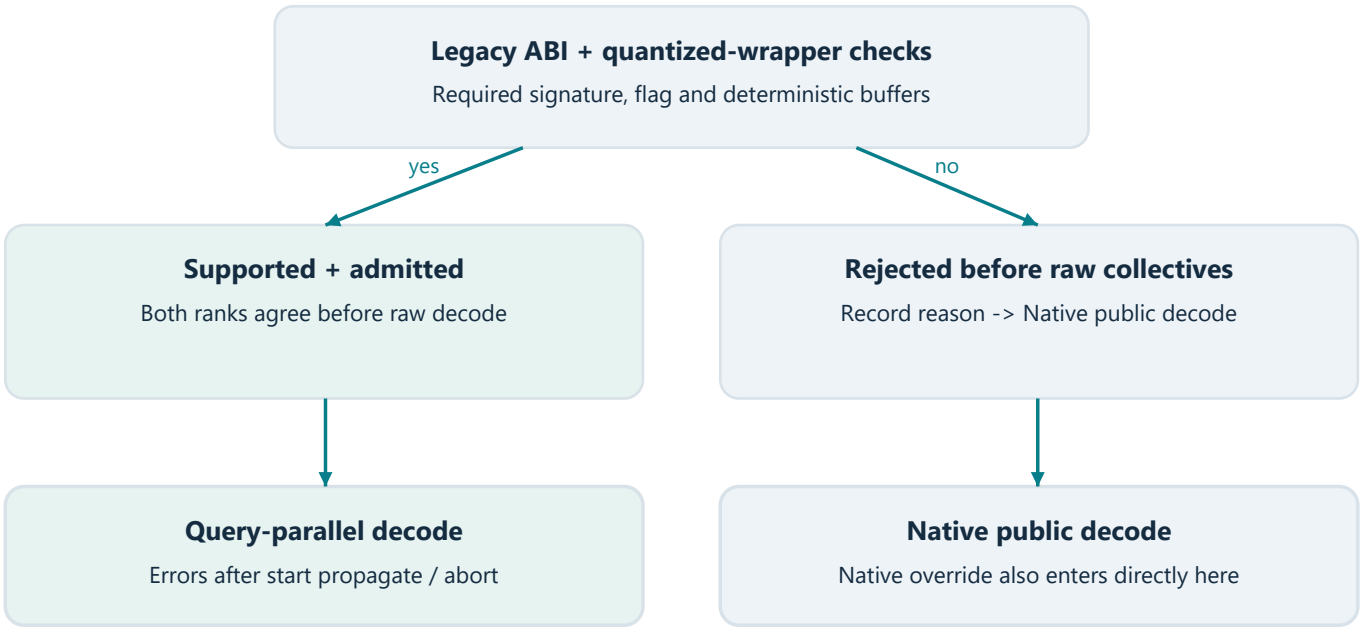
> **Why not independent video slabs?**
> Decoding cropped spatial/temporal pieces independently would remove global attention context and can alter boundaries or consistency. This implementation partitions queries while retaining full KV. It is not head sharding or independent cropped-slab generation.

The parent replaces only the raw _decode_pixels boundary. Native normalization, temporal chunking, spatial tiles, blending, trimming, public output buffers and finalization remain in place. Decoder replicas are reused across raw calls within one public decode and released afterward.

Source: S05, S25, S36

Compatibility is checked before collectives

The old attention API is preserved because it is part of the qualified contract.



Query partitioning requires the Native Kitchen dispatch flag, quantized QKV wrappers, qk_norm_scale and the legacy attention signature accepting pre_norm, residual and residual_scale. Package import success alone does not prove those contracts. The staged decoder rebuilds nonpersistent buffers that are absent from safetensors.

The parent loads its required stage before VRAM admission so that allocation is visible. Both ranks agree with a MIN collective on conservative host/GPU admission before raw decode starts. Decode mode native bypasses parallel preparation. Very short sequences without two aligned ranges run Native raw decode on rank 0.

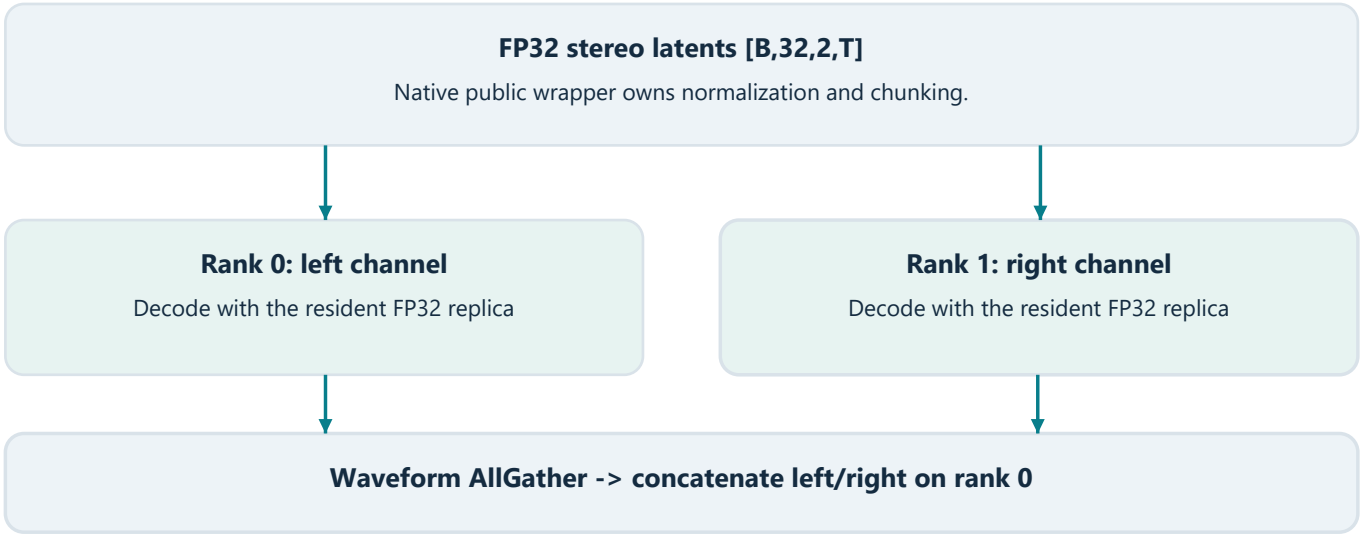
> **Two different failure boundaries**
> A compatibility or admission rejection can select Native public decode before raw collectives. An error after collective decode begins must propagate and abort/clean up the service; silently switching one rank to Native could deadlock or accept a partial output.

The sibling ComfyUI-master has a different attention wrapper. The pinned checkout retains the required API. A changed wrapper can disable query parallelism even with the same Kitchen package version. The solution here is source/version preservation and a recorded fallback reason, not an automatic upstream API migration.

Source: S24, S25, S28, S29, S35, S36, S39

Two channels, two FP32 decode owners

Channel independence creates parallel work without temporal tiling.



The active worker path is audio_channel_decode.channel_decode. It accepts FP32 stereo latents shaped [B,32,2,T]. Rank 0 decodes channel 0, rank 1 decodes channel 1, using the already-loaded decoder replica. A waveform AllGather restores left/right order; rank 0 concatenates the channels for the Native public wrapper.

The public VAE continues to own chunking, normalization and output layout conversion. The implementation does not construct a second dual-device manager inside each persistent worker, and it does not import the historical temporal/tiled benchmark manager.

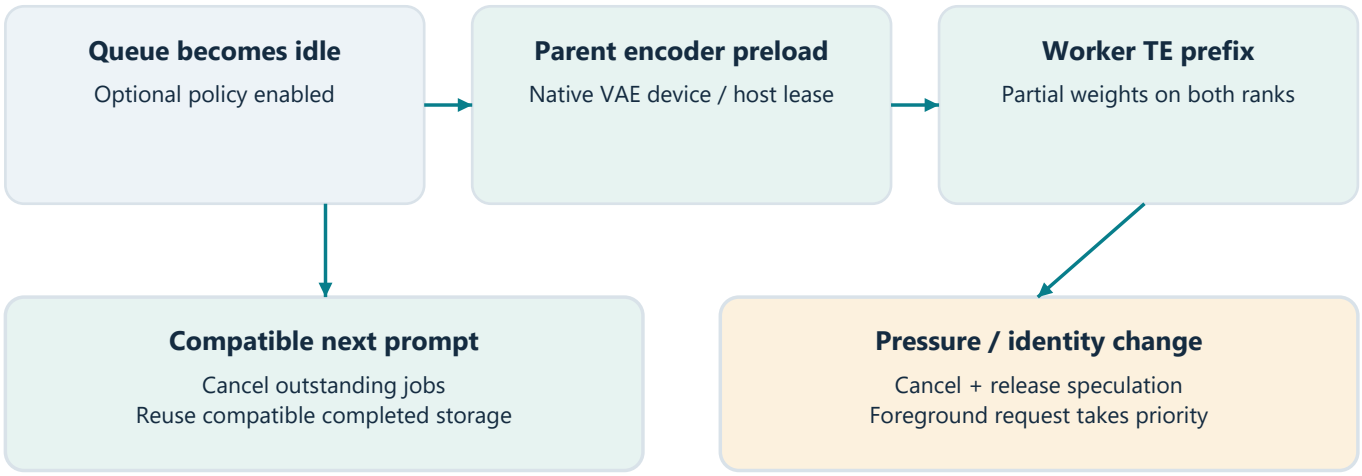
Numerical comparison	Recorded result / interpretation
Sequential mono vs channel ownership	Exact in the retained local qualification.
Native stereo vs split mono	Relative L2 approximately 1.280e-5, 1.237e-5 and 1.205e-5 at 5/10/20 seconds. The old strict 1e-5 gate still fails.
Perceptual/audio claim	No claim of inaudibility follows from those metrics. Native decode remains selectable.

Accepted approximation, not renamed exactness
Splitting the channel/batch geometry can change the FP32 convolution/backend reduction choices. The weights and mathematical channel independence remain, but floating-point evaluation is not guaranteed byte-identical to Native stereo. This difference is explicitly retained and disclosed.

Source: S05, S25, S26, S36

Preload encoders, then a bounded TE prefix

Speculation is optional, cancellable and tied to the next compatible request.



No speculative prompt inference or decoder computation is performed.

With idle_residency=True, successful queue-idle completion starts a background task for the registered VAE encoders, then asks both workers to begin partial TE preload. Encoders live in the parent on the ordinary Native VAE device, normally GPU 0. They are not replicated on the two TP ranks, and workers see their allocation through driver-memory admission.

Encoder reads check cancellation between tensors. GPU preload needs weight storage plus the configured idle reserve. If GPU headroom is insufficient, a prepared CPU encoder may still be reused. If encoder speculation fails, the reason is recorded and TE speculation may still proceed. No reference encoding, prompt conditioning, initial noise or decoder inference is executed speculatively.

Event	Lifecycle action
New prompt	Cancel outstanding speculation; reuse only compatible completed entries.
Active graph changed	Filter registered VAE names and release stale cached nodes.
Memory pressure / incompatible identity	Release encoder and TE speculation; foreground work has priority.
Denoising starts	Explicitly release staged encoders; active GPU window takes ownership.

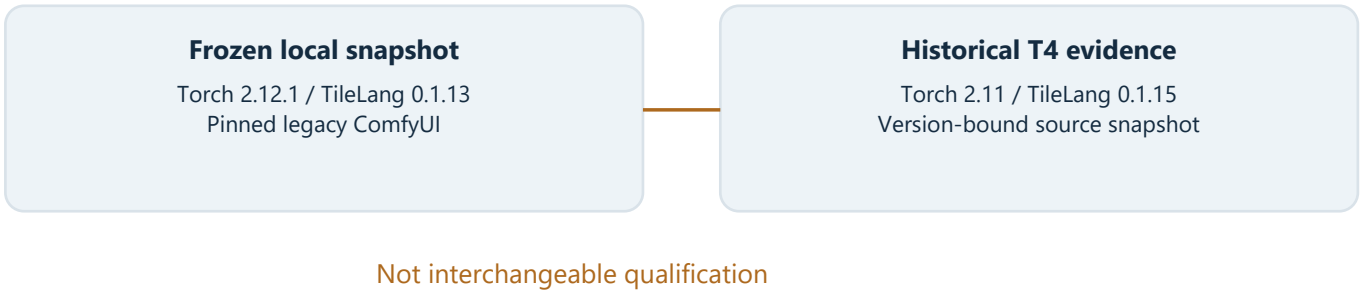
Default idle weight cap is 4 GiB/rank, with a 4 GiB idle reserve and 0.25-second delay; idle remains disabled by default. This is separate from the active 6 GiB managed GPU weight cap. Reuse never bypasses source/adapter/operator identity checks or stream-lifetime fences.

Source: S02, S03, S23, S24, S27

The validated runtime is an input

Operator adaptation must not silently upgrade source, APIs or dependencies.

Contract	Preserved local stack
ComfyUI source	d49e888586dd8ae012c0667b33466b815fee07f7
Default interpreter	C:/ProgramData/miniconda3/envs/comfyui/python.exe
Torch / Kitchen / TileLang	2.12.1+cu132 / 0.2.37 / 0.1.13
Frozen environment record	runtime/local_conda.json
Historical physical T4 operator stack	Torch 2.11.0+cu130, Kitchen 0.2.37, TileLang 0.1.15; nvcc 12.8.93
Current Linux qualification	Not established by the historical stack. Explicit platform lock / ABI work remains.



The launcher selects the pinned local ComfyUI and checks normalized source identity. It does not replace it with the API-different sibling checkout. The original sibling has no Git identity in the retained provenance record; this document does not claim it was synchronized upstream or rolled back.

Kaggle jobs must use the frozen source archive and exact package versions. Linux-only CUDA/NCCL/Triton dependencies and wheel/ABI differences require explicit resolution and validation. Current setup blocks rather than substituting versions. The historical TileLang 0.1.15 gate does not automatically qualify 0.1.13 on Linux.

> **No hidden environment switch**
> The historical D:/h3_tp2_legacy_env is not the default. This PDF is generated with the separate bundled document runtime; it does not install or change inference packages. No GPU time is consumed for document creation.

Source: S28, S29, S33, S39

What the retained checks actually establish

Conformance, physical transport and full-model quality are separate evidence classes.

Evidence	What it covers	What it does not cover
Pipeline review: 56 tests	Lifecycle, source/runtime preservation, policy/UI, loading, cache identity, debug isolation	Full H3 generation speed or physical NCCL timing
Historical physical integration: 10 gate records/rank	Masked GQA, selected B ownership, drop/scalar dispatch, native INT32 row collectives	Current frozen Linux stack; full-depth quality
Enhancements: local arithmetic / contracts	Scaled-FP16 codec reference, refiner SDPA, positional-read fallback, explicit lifetimes	New physical T4 performance for those final sources
FC2 physical v1/v2	Trained owned-rotation payload/scale/output identity; scoped memory/latency; integrated S32768 dispatch	Universal GEMM rounding equivalence or pipeline VRAM
Trained VAE local checks	36-block video query and staged encode/decode agreement in tested fixtures; audio differences recorded	Physical integrated near-capacity two-T4 request performance

The charts in this guide use retained measurements and do not launch new experiments. Historical sources/results stay version-bound. A test against an approximate reference can pass perfectly while still differing from dense attention or Native INT32 transport.

Nsight Systems evidence in prior operator investigations explains launches, preparation, collective intervals and allocation boundaries. Nsight Compute returned ERR_NVGPUCTRPERM on the retained Kaggle probes. Registers/shared-memory sizes and kernel interval coverage are not substitutes for achieved occupancy, SM utilization, memory-stall or Tensor Core counter measurements.

> **Full-pipeline scope**
> The complete current default/optional dispatch combinations have not all been qualified for full-depth two-T4 generation, temporal/audio semantics, queued concurrency or end-to-end latency. Operator gains must not be added together into a pipeline speedup.

Source: S30-S39

Operate, inspect and change one boundary at a time

Use the explicit configuration and preserve Native fallbacks.

```
from patches.comfy_tp_runtime import RuntimeConfig
from patches.h3_operator_policy import H3OperatorPolicy
config = RuntimeConfig(
    te_path, dit_path,
    adaptive_lookahead=False,
    operators=H3OperatorPolicy(
        te_attention="tilelang_group32",
        dit_attention="kitchen",
        row_linear_backend="tilelang_packed",
        fc2_quantizer="owned_fp32",
        swiglu_padding=True, selective_b=True,
        scalar_score_workspace_mib=96,
    ),
)
```

Use the linked MiniMaxH3TP2Loader and parallel VAE nodes so that all stages share one persistent session. A changed policy or checkpoint/adapter identity recreates the service. Production TileLang dispatch checks SM75; an SM120 arithmetic preview must not be reported as a target run. Physical TP2 requires matching CUDA devices and an available NCCL backend.

Symptom / investigation	First boundary to inspect
Unexpected Native fallback	Selected source identity, GPU capability, VAE attention ABI and explicit decode_mode.
High peak memory	Live aliases, parent VAE plus worker replicas, FC1/O stage peaks, weight/pinned/host budgets.
No sparse speedup	Preparation/routing/summary costs and complete-API timings; never only omitted-block count.
Quality drift	Selected approximation: sparse mode, FP16 SUM, Row Partial or offline requantization. Hold inputs/noise fixed.

Runtime code is under patches/ and the custom node. Read-only stats snapshots and bounded counters are separate from explicit profilers/captures in benchmarks/. Native reference loaders are lazy qualification paths. Legacy PP/SP/tiled interfaces remain compatible but are not selected by this TP2 loader.

> **Not integrated by this document**
> TE compression codecs, experimental attention INT8 PV, packed-INT8 transport and broad auto-tuning are not enabled. No H3 inference, dependency setup job, new profiler capture or Git publication was performed to produce this guide.

Source: S01-S03, S28, S33-S39

Implementation source catalogue

Paths are relative to the root on the cover. The PDF embeds a JSON manifest with source hashes.

S01 patches/h3_operator_policy.py
Operator defaults, architecture guards and refiner SDPA

S02 custom_nodes/minimax_h3_tp2/__init__.py
Node controls and service identity

S03 patches/comfy_tp_runtime.py
Persistent workers, runtime budgets and phase protocol

S04 patches/comfy_tp_models.py
Native model construction and staged startup

S05 patches/comfy_sampler.py
Native conditioning/sampling and public decode adapters

S06 patches/tp_qwen3vl.py
TE TP2 geometry, attention boundary and tensor lifetimes

S07 patches/tp_dit.py
DiT TP2 geometry, AdaLN and residual order

S08 patches/tilelang_group32_masked.py
Mask-aware Group32 TE kernel

S09 patches/tilelang_grouped_tp_kitchen_v2.py
Dense preparation, query caching and kernel mapping

S10 patches/tilelang_grouped_tp_optimized.py
Sparse preparation, tau0 routing and reference equations

S11 patches/tilelang_attention_p2.py
Omitted-only centroid score workspace

S12 patches/tilelang_row_quant.py
Global-MAX activation quantizer and packed row preparation

S13 patches/tilelang_swiglu_pad.py
SwiGLU fusion, lifetime repair and owned FP32 rotation

S14 patches/tilelang_transport_p2.py
Optional FP16 transport and fused dequantization

S15 patches/tilelang_transport_tradeoffs.py
Immutable transport codec and power-of-two bound

S16 patches/parallel_int8.py
INT8 GEMM, INT32 SUM and native dequantization reference

S17 patches/optimized_tp.py
Native/SM75 rotation dispatch

S18 patches/h3_lora.py
Ordered online adapters, selective B and row partials

S19 patches/offline_lora.py
Whole-layer dual-GPU baking and atomic publication

S20 patches/windowed_tp.py
Positional reads, streamed specs and model proxies

Source: Current source audit: 2026-10-20

Evidence and lifecycle source catalogue

Read code for current selection; read version-bound results for measurement scope.

S21 patches/rank_local_int8.py
Owned checkpoint ranges and optional rank-local exports

S22 patches/host_weight_cache.py
Shared host budget, pageable cache and pin staging

S23 patches/weight_residency.py
H2D events, leases, eviction and adaptive lookahead

S24 patches/staged_vae.py
Encoder/decoder-only construction and nonpersistent buffers

S25 patches/parallel_vae.py
Video query ownership, ABI gates and decoder replicas

S26 patches/audio_channel_decode.py
Stereo FP32 channel ownership and final AllGather

S27 patches/comfy_idle_lifecycle.py
Queue-idle admission, pressure and cancellation hooks

S28 patches/legacy_runtime.py
Pinned source selection and source identity checks

S29 runtime/local_conda.json
Frozen user environment and ComfyUI identity

S30 docs/reports/FC2_QUANTIZER_MEMORY.md
FC2 memory study, qualification scope and activation defaults

S31 docs/data/fc2_quantizer_t4_v1_summary.json
Physical T4 FC2/FFN measurements at S513/8192/16384

S32 docs/data/fc2_quantizer_t4_v2_summary.json
Integrated physical T4 S32768 confirmation

S33 docs/guides/H3_OPERATOR_INTEGRATION.md
Historical target gates and integration boundaries

S34 docs/guides/H3_OPERATOR_ENHANCEMENTS.md
Transport/lifetime/read/refiner enhancement qualification

S35 docs/guides/H3_LOADING_STAGES.md
LoRA bake, VAE component read sizes and idle encoder stages

S36 docs/guides/PARALLEL_VAE_INTEGRATION.md
Public decode preservation and audio numerical limits

S37 docs/data/H3_PIPELINE_REVIEW_RESULTS.json
Recorded pipeline review checks and source closure

S38 docs/reports/TP2_P2_OPTIMIZATION_REPORT.md
Historical profiling, scalar workspace and transport caveats

S39 docs/reports/H3_RUNTIME_PROVENANCE_AUDIT.md
Local source provenance and Linux qualification limits

Source: Current source audit: 2026-10-20